

5

NOSQL Data Management for Big Data and Visualization

Syllabus

NoSQL Databases : Schema-less Models : Increasing Flexibility for Data Manipulation-Key Value Stores- Document Stores - Tabular Stores - Object Data Stores - Graph Databases Hive - Sharding -- Hbase – Analyzing big data with twitter - Big data for E-Commerce Big data for blogs - Review of Basic Data Analytic Methods using R.

Contents

- 5.1 Introduction to NoSQL
- 5.2 "Schema-Less Models" : Increasing Flexibility for Data Manipulation
- 5.3 Key Value Stores
- 5.4 Document Stores
- 5.5 Tabular Stores
- 5.6 Object Datastores
- 5.7 Graph Datastores
- 5.8 Hive
- 5.9 Sharding
- 5.10 Hbase
- 5.11 Analyzing Big Data with Twitter
- 5.12 Big Data for E-Commerce and Blogs
- 5.13 Review of Basic Data Analytic Methods using R

Summary

Two Marks Questions with Answers [Part - A Questions]

Part - B Questions

5.1 Introduction to NoSQL

With Rapid development in digital technologies, the massive amount of data is being generated which has four Big data characteristics like volume, variety, veracity and velocity. Earlier, the relational database management system was used to perform all database operations. The RDBMS databases used to have SQL databases which used to store data in to table based databases. It has rows to store records and columns to define the attributes. Each row of table contains a unique instance of data for the categories defined by the columns along with primary key, to uniquely identify the rows. In traditional analytics system, the cleansed or meaningful data were collected and stored by RDBMS in a data ware house. That data was analyzed by means of performing Extract, Transform and Load (ETL) operations. But, because of above four Big data characteristics it becomes more and more difficult to capture, store, organize, process and analyze the data generated by various web applications or websites. Therefore, organizations were faced many limitations related to traditional relational database management system which are listed below

- a) It has support of only structured and stored data
- b) It required data to be in table form
- c) The database must have predefined schema with limited in size
- d) It does not support the scalability
- e) Parallel processing of such data by traditional analytics were costlier because of expensive hardware.
- f) Unstructured and semi structured data like images, pdfs, compressed files, videos etc. was not supported
- g) Realtime data capturing and analytics was not supported
- h) Most of the databases were proprietary which incurs huge cost for storing and processing data

Therefore, to overcome above challenges the special kind of database was invented called NOSQL. The NoSQL database stands for "Not Only SQL" or "Not SQL." The NoSQL is a set of concepts that allows the rapid and efficient processing of data sets with a focus on performance, reliability, and agility. It does not require a fixed schema, normalized data (3NF), tables or joins, instead it is used for distributed data stores with easier scalability. It is largely used for Big data applications where real-time data is

generated along with huge volume, variety, velocity and veracity. For example, social media websites like Twitter, Facebook, Google, Instagram etc. that collect and process terabytes of user's data every single day who uses NOSQL databases for their all the operations.

The NOSQL databases came in to picture which has many advantages over the traditional RDBMS. The major advantages of NOSQL databases over traditional SQL databases are given as below

- a) It supports real time or batch processing data along with different formats like structured, unstructured or semi-structured.
- b) It can process uncleaned or uncertain data for analytics
- c) It does not require an expensive hardware for implementation as it can be run on commodity hardware
- d) It supports huge volume of data generated real time with velocity
- e) It can perform data analytics at low cost.
- f) It is free of joins and schema and doesn't require complex queries
- g) It can work on many processors parallelly with linear scalability
- h) Most of NOSQL systems are opensource that makes it very cost efficient
- i) Cost per bit for processing is very low and compatible with cloud computing

5.1.1 Types of NoSQL Databases

There are five basic types of NoSQL databases :

1. **Key-Value Store** : It uses combination of keys and values stored in big hash tables to process the data. e.g. Redis, Riak, Amazon DynamoDB etc.
2. **Document Store** : It store and process data in terms of documents in a collection. e.g. MongoDB, CouchDB, Amazon DocumentDB etc.
3. **Tabular Store (Column-based Store)** : These databases are capable of storing and processing data in a column of a table. Some of the columnar databases are Hbase, Cassandra, Azure Table Storage (ATS), BerkeleyDB etc.
4. **Object Store** : These types of databases stores object in a database with capability to represent ORDBMS. The examples of Object stores are ObjectDB, Perst, Objectstore etc.

5. **Graph-based databases** : This type of databases are network databases that uses edges and nodes to represent the stored data with relationship. e.g. Apache Giraph, GraphX, Neo4j etc.

The above NOSQL databases are explained briefly in subsequent sections of this chapter

5.2 "Schema-Less Models" : Increasing Flexibility for Data Manipulation

NoSQL data systems hold out the guarantee of flexibility and more noteworthy adaptability in database management while lessening the reliance on progressively formal database administration. The NoSQL databases have increasingly loosened up modeling constraints, which may benefit both the application engineer and the end user's analyst when their intuitive analysis are not throttled and required to cast each query in relational table based environment. Therefore, different NOSQL databases are enhanced for various kinds of analysis. For instance, some are implemented as key value stores, which pleasantly adjust to certain big data programming models, while another rising model is a graph database, where a graph abstraction is implemented to insert both semantics and connectivity inside its structure. In fact, the general ideas for NoSQL incorporate schema less modelling in which the semantics of the data are inserted inside an adaptable connectivity and storage model; this accommodates automated distribution of data and scalability with respect to utilization of compute, storage and network for data transmission in manners that don't drive specific binding of data to be persistently stored in physical locations. NoSQL databases additionally accommodate integrated data caching that makes a difference in reducing the data latency and speedy execution.

5.3 Key Value Stores

A relatively simple type of NoSQL data store is a key value store, a schema less model in which values are related with particular character strings called keys which are stored in a hash table. It is presented with a simple string called key that returns an arbitrary large BLOB of data called value. It doesn't have query language which provides a way to add and remove key-value pairs into/from a database. Its store is like a dictionary which has a list of words and each word has one or more definitions. The key value type basically, uses a hash table in which there exists a unique key and a pointer to a particular item of data. Key-Value is based on a hash table where there is a unique key and a pointer to a particular item of data. The example of key value store is shown in Fig. 5.3.1.

	Key	Value
Image Name →	image-12345.jpg	Binary image file
Web page URL →	http://www.example.com/my-web-page.html	HTML of a web page
File path name →	N:/folder/subfolder/mylife.pdf	PDF document
MD5 hash →	9e107d9d372bb6826bd81d3542a419d6	The quick brown fox jumps over the lazy dog
REST web service call →	view-person?person-id=12345&format=xml	<Person><id>12345</id> </Person>
SQL query →	SELECT PERSON FROM PEOPLE WHERE PID = "12345"	<Person><id>12345</id> </Person>

Fig. 5.3.1 : Example of key value store

Let us take another example of country and locality represented in Table 5.3.1 where name of the country is the key, while locality (addresses) in that country is value.

Key	Value
"India"	{"C/09, Shastrinagar, Maharastra, India - 400050"}
"Romania"	{" City Business Center, Coriolan Brediceanu No. 10, Building B, Casario, 30211"}
"US"	{" Ridge Drive. Suite 200 South, Fairfax, California 22033"}

Table 5.3.1 : Example of Key-value store for country database

The key value pair store does not impose any constraints about data typing or data structure instead used for consuming business applications to assert expectations about the data values and their semantics with interpretation which schema-less property of the model. The different operations can be performed over key value store are given in Table 5.3.2 and explained in Fig. 5.3.2. The easiness of the representation in key value data store allows massive amounts of indexed data values to be added to the same key value table, which can then be sharded, or distributed across the storage nodes.

Sr. No.	Function	Description
1.	Get(key)	Returns the value associated with the provided key.
2.	Put (key, value)	Which adds the value associates with the key.
3.	Multi-get (key1, key2,..., keyN)	Returns the list of values associated with the list of keys.
4.	Delete(key)	Removes the entry for the specified key from the data store.

Table 5.3.2 : Different operations in key value store

Under the right conditions, the table is circulated in a way that is aligned with the way the keys are sorted out so that the hashing function is utilized to figure out which node holds that key's bucket While key value sets are extremely valuable for both storing the results of analytical algorithms and for creating those outcomes for reports.

Some of the popular use cases of the key-value databases are

- Storing and processing user session data
- Maintaining schema-less user profiles
- Storing user preferences
- Storing shopping cart data

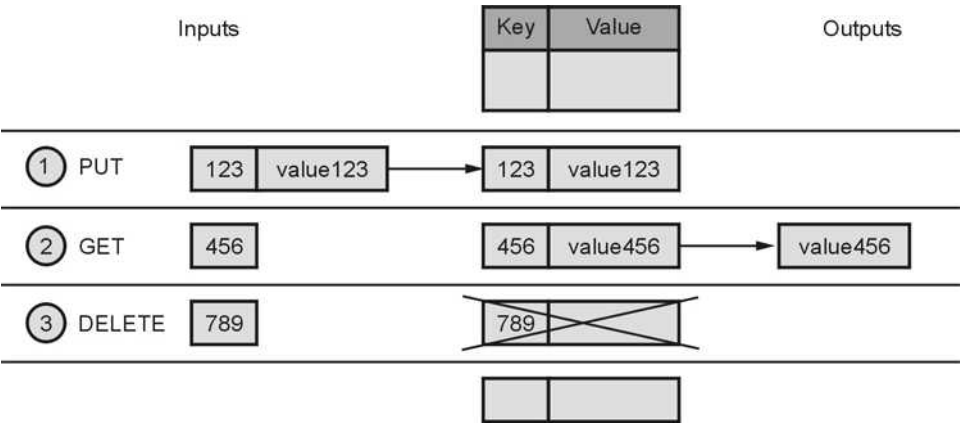


Fig. 5.3.2 Key value datastore operations

One is that the model won't naturally give any sort of conventional database abilities (such as atomicity of transactions, or consistency when numerous transactions are executed at the same time) those abilities must be given by the application itself. Another is that as the model develops, keeping up unique values as keys may become more difficult, requiring the introduction of some complexity in creating character strings that will stay novel among a myriad of keys.

5.4 Document Stores

The document-based NoSQL database stores and retrieves the data as a key value pair where value part is stored as a document with associated key. In document store each record is considered as a separate document where everything inside a document is automatically indexed when a new document is added. Although the indexes are large

and everything is searchable within the document, most of the document stores group documents together in a collection. The document that has some structure and encoding for managing the data. Some of the common encoding used with the document stores are XML, JSON (Java Script Object Notation), BSON (which is a binary encoding of JSON objects) etc. or other means of serializing data. The document representation embeds the model so that the meanings of the document values can be inferred by the application. It is not suitable for running complex queries or if the application requires complex multiple operation transactions.

The difference between RDBMS and document store database is given Table 5.4.1.

RDBMS	Document store
Database	Database
Table	Collection
Tuple/Row	Document
Column	Field
Primary key	Primary key (Default key <code>_id</code> provided by mongodb itself)

Table 5.4.1 : RDBMS vs Document store database

The commonly used use cases of document store databases are E-commerce platforms, content management systems, web analytics, analytics platforms or blogging platforms. One of the differences between a key value store and a document store is that the key value store requires the use of a key to retrieve data, while document store often provides an object metadata used for querying the data based on the contents either through a programming API or using a query language.

The following example shows the insertion of data values and respective keys in to a document called Book. It has keys like id, title, description, by, array of tags and likes. The respective values for keys are assigned at right hand side.

```
db.Book.insert(
{
  "_id": ObjectId(7df78458902c),
  "title": "Python Programming",
  "description": "Python is a Object oriented language ",
  "by": "Technical publication",
  "tags": ["SciPy", "NumPy", "Pandas"],
  "likes": "100"
})
```

5.5 Tabular Stores

Tabular, or table-based stores are to a great extent dropped from Google's unique Bigtable design to manage structured and unstructured data. The HBase is an example of Hadoop integrated tabular NoSQL database which is evolved from big tables. The big table NoSQL model enables to sparse data which is to be stored in a three-dimensional table that is indexed by row key, column key that indicates the attribute for which data value is stored, and a timestamp that may allude to the time at which the rows column was stored.

For example, different attributes of a web page can be related with the page's URL as the HTML substance of the page, URLs of other website pages that connect to this web page, and the creator of the web page. column in a Bigtable model are assembled as "families," and the timestamps empower the management of multiple versions of an object. Each time the content changes, a new column affiliation can be made with the timestamp of when the content was downloaded.

The column-oriented NoSQL database is the one of the popular tabular databases which is shown in Fig. 5.5.1. They are widely used in applications like business intelligence, CRM, Library card catalogs and in data warehouse.

ColumnFamily			
Row Key	Column Name		
	Key	Key	Key
	Value	Value	Value
	Column Name		
	Key	Key	Key
	Value	Value	Value

Fig. 5.5.1 Column based tabular datastore

In Column based Tabular datastore, the ColumnFamily is used as a single structure that can group Columns. The Key attribute is the permanent name of the record that can have different numbers of columns and values that can be scaled in an irregular way. The Value attribute represents the description about the keys whereas Column maintains an ordered list of elements as known as tuple with a name and a value defined in it.

5.6 Object Datastores

The Object data store stores and manage the different kinds of objects in a database. Somehow or another, object data stores and object databases appear to bridge the schema-less data management and the traditional relational models. The object databases are similar to document stores except document stores expressly serializes the objects so the data values are stored as a string. While object databases keep up the object structures as they are bound to object-oriented programming languages like C11, Objective-C, Java, and Smalltalk. On other hand, object database the board frameworks are bound to give customary ACID (atomicity, consistency, isolation, and durability) compliance. Object databases are not relational databases and cannot be queried using SQL. It is mostly used in cloud computing to store files as an object.

5.7 Graph Datastores

The Graph stores are highly optimized to proficiently store graph nodes and links, and enable you to query these graphs. The Graph databases are valuable for any business issues that has complex relationships between objects such as social networking, rules-based engines, creating mashups, analyzing complex network structures and finding patterns within these structures.

The graph store has three data fields namely nodes, relationships, and properties as shown in Fig. 5.7.1. The Graph stores are mainly used for identifying the distinct patterns of connections between nodes. The Graph databases provide a model of representing discrete entities and many kinds of relationships that links those entities.

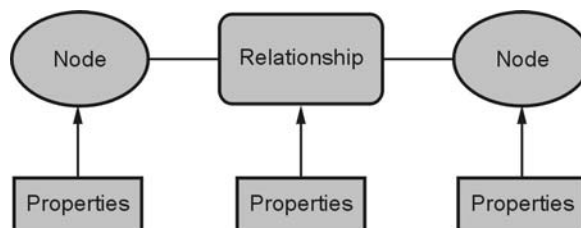


Fig. 5.7.1 Data fields in Graph datastore

The graph datastore of movies with different actors, properties and relationship are shown in Fig. 5.7.2.

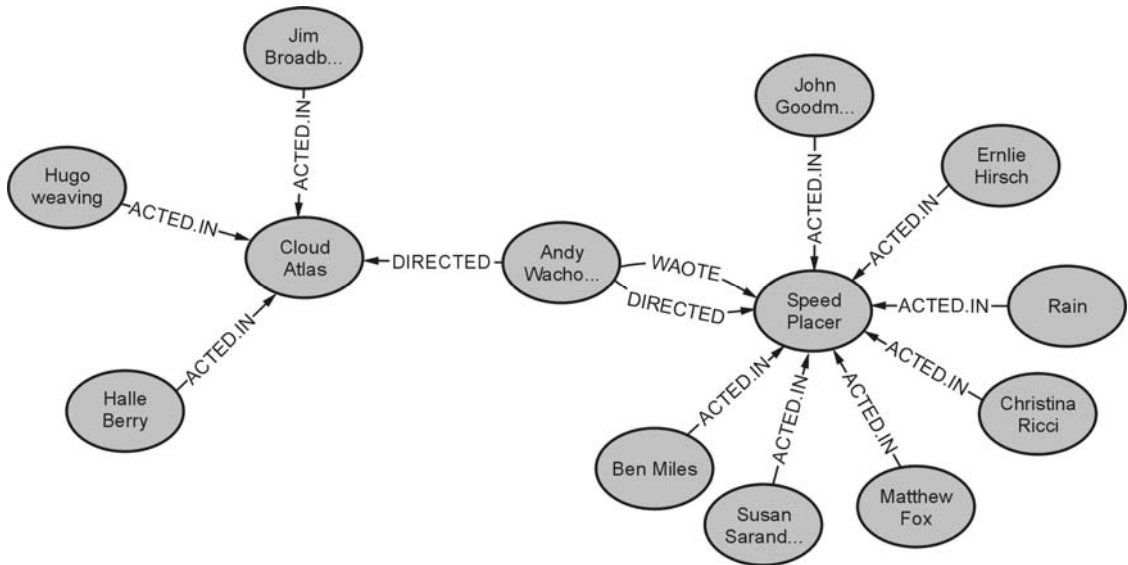


Fig. 5.7.2 Graph of movies database

In above graph, the actors (like Emile Hirsh, Rain, Halle Berry etc.) at leaf nodes represented by leaf nodes which are connected to the properties like movies and relationship between actors and movies are defined by arrows.

5.8 Hive

The Apache Hive is a data warehouse software built on the top of Hadoop that facilitates reading, writing, and managing large datasets stored in HDFS. The data stored by Hive may residing on distributed storage which can be queried using SQL. It is mainly used for storing the data in a data warehouse and performs different SQL operations over that. The hive provides a simple SQL-like query language called Hive Query Language (HQL) for querying and managing the large datasets. Hive engine compiles these queries into Map-Reduce jobs to be executed on Hadoop.

The basic features of hive are given as follows

- It stores schema in a database and processed data into HDFS.
- It provides SQL type language for querying called HiveQL or HQL.
- It is fast, scalable and extensible.
- Multiple users can simultaneously query the data using Hive-QL.
- It supports different data formats with real time data analytics
- It can store and process large datasets in data warehouse

Using the Hive query language (HiveQL), which is very similar to SQL, queries are converted into a series of jobs that execute on a Hadoop cluster through MapReduce or Apache Spark. The Users can run batch processing workloads with Hive as well as can analyze the same data simultaneously using interactive SQL or using machine-learning workloads over the tools like Apache Impala or Apache Spark within a single platform. The various SQL commands used in hive are given in Table 5.8.1.

Sr. No.	Function	Command
1	Create database	hive> create database dbname;
2	View databases	hive> show databases;
3	Select database	hive> use dbname;
4	Copy input data to HDFS from local	hive> hadoop dfs -copyFromLocal <local path to file> hdfs:/
5	Create table	hive> CREATE TABLE <tablename> (attribute-1 datatype, attribute-2 datatype, -attribute-n datatype);
6	Copy local data in Hive table	hive> LOAD DATA INPATH <filename> OVERWRITE INTO TABLE <tablename>
7	Insert data	hive> INSERT INTO [TABLE] <tablename> [(column_list)] [PARTITION (partition_clause)]
8	Retrieving all the values	hive> SELECT * FROM <tablename>;
9	Update data	hive> UPDATE <tablename> SET [column = value ...] [WHERE expression]
10	Delete Touple	hive> DELETE FROM <tablename> [WHERE expression]

Table 5.8.1 : SQL commands used in hive

It has support for different aggregation functions of SQL like SUM, COUNT, MAX, MIN etc. and other functions like CONCAT, SUBSTR, ROUND etc. It also supports GROUP BY and SORT BY clauses along with joins. It comes with a command-line shell interface which can be used to create tables and execute queries. In addition, custom Map-Reduce scripts can also be plugged into queries.

As Hive is a petabyte-scale data warehouse system built on the top of Hadoop platform, it allows programmers to write the custom Map-Reduce framework to perform more sophisticated analysis and data processing. Therefore, Hive on MapReduce or Spark is best-suited for batch data preparation or ETL.

5.9 Sharding

The Sharding is a type of database partitioning that splits very large databases the into smaller, faster, more easily managed parts called data shards. The word shard means a small part of a whole. It is a database architecture pattern related to horizontal partitioning that makes separation of one table's rows into multiple different tables, known as partitions. Each partition has the same schema and columns, but also entirely different rows. Likewise, the data held in each is unique and independent of the data held in other partitions.

Original Table			
Cust_id	Name	Gender	City
1	Mukesh	Male	Mumbai
2	Arun	Male	Delhi
3	Bhushan	Male	Chennai
4	Sonali	Female	Hyderabad

Horizontal Partition 1			
Cust_id	Name	Gender	City
1	Mukesh	Male	Mumbai
2	Arun	Male	Delhi

Horizontal Partition 2			
Cust_id	Name	Gender	City
3	Bhushan	Male	Chennai
4	Sonali	Female	Hyderabad

Vertical Partition 1		
Cust_id	Name	Gender
1	Mukesh	Male
2	Arun	Male
3	Bhushan	Male
4	Sonali	Female

Vertical Partition 2	
Cust_id	City
1	Mumbai
2	Delhi
3	Chennai
4	Hyderabad

Table 5.9.1 : Horizontal and vertical partitioning

In a vertically-partitioned table, entire columns are separated out and put into new, distinct tables. The data held within one vertical partition is independent from the data in

all the others, and each table holds both distinct rows and columns. The Table 5.9.1 illustrates how a table could be partitioned both horizontally and vertically.

The Sharding involves breaking up the data into two or more smaller chunks, called logical shards. The logical shards are then distributed across separate database nodes, referred to as physical shards, which can hold multiple logical shards. Despite this, the data held within all the shards collectively represent an entire logical dataset.

The major benefits of sharding are, it can help to facilitate horizontal scaling easily, it helps to speed up query response times and it help to make an application more reliable by mitigating the impact of outages. While the drawbacks of Sharding are complexity of properly implementing a sharded database architecture, mostly shards eventually become unbalanced and it becomes very difficult to return back to its unsharded architecture once sharding is done.

5.10 Hbase

Hbase is an column-oriented non-relational database management system that runs on the top of Hadoop Distributed File System (HDFS) and used to store and process unstructured data. It provides a fault-tolerant way of storing and processing large data sets and is well suited for real-time data processing or random read/write access to large volumes of data. It does not support a structured query language like SQL and relational data stores at all. The HBase applications are usually written in Java like a typical MapReduce application. It is designed to scale linearly and comprises a set of standard tables with rows and columns, much like a traditional database. Each table in a Hbase must have an element defined as a primary key to uniquely define the attribute. It supports a rich set of primitive data types like numeric, binary and strings along with complex types including arrays, maps, enumerations and records. The HBase mostly relies on Zookeeper for getting high-performance coordination which is built into Hbase. It is also compatible with Hive which gives query engine for batch processing of big data and fault-tolerance in big data applications.

The basic features provided by Hbase are listed as follows

- It offers consistent reads and writes on stored and real time data
- It provides atomic Read and Write where during one read or write process, all other processes are prevented from performing any read or write operations
- It offers database sharding of tables which is required to reduce I/O time and overhead
- It provides high availability, high throughput along with scalability in both linear

and modular form

- It supports distributed storage like HDFS
- It has built-in support for failover along with backup
- It supports real time data processing using block cache and Bloom filters to make real time query processing easier
- It supports data replication across clusters.

The Hbase has Hbase shell to execute the queries. The commonly used commands (queries) in Hbase are described in Table 5.10.1.

Sr. No.	Function	Command
1	Listing the database	hbase(main):003:0> list
2	Load databases	hbase(main):012:0> scan '<table_name>'
3	Add a tuple	hbase(main):004:0> put <table_name>, <Row_no>, <ColumnFamily> <value>
4	Retrieve a tuple	hbase(main):023:0> get <table_name>, <Row_no>
5	Create table	hbase(main):002:0> create <table_name>, <ColumnFamilyName>
6	Delete table	hbase(main):003:0> drop <table_name>

Table 10.1 : Hbase commands

5.11 Analyzing Big Data with Twitter

Twitter (www.twitter.com) is an online social networking web platform that allow users to send and read short 140-character text messages, called "tweets". It has more than 250+ million monthly active users who send about 500+ million tweets per day and as per Alexa's web traffic analysis, it is ranked amongst the top 10 most visited websites in the world. The twitter shares its data in a document store format (mostly in JSON - JavaScript Object Notation) and allows developers to access that data using one of the most secured authentication APIs like OAuth. OAuth provide a safer alternative because the user generally logs into twitter and approves sharing of his information to each application. Once we have received data from twitter we store it in NOSQL database over which Map-Reduce approaches can be applied to analyze the same. To analyze such a massive data relational database are not enough therefore massively parallel and distributed systems are required like Hadoop.

There are two kinds of analysis that can be performed over the twitter data. The trending analysis allows to analyze individual tweets and looks for certain words amongst them while sentiment analysis allows to look for certain keywords in a tweet

and analyzing them to compute a score of sentiments. The sentiment analysis is the process of detecting the contextual polarity of text. A common use case for sentiment analysis is to discover how people feel about a particular topic.

5.11.1 Sentiment Analysis using Hadoop

The technologies that are required to perform sentiment analysis are Apache Flume, Hive and Pig. The Apache Flume is a distributed, reliable, and available service for efficiently collecting, aggregating, and moving large amounts of streaming data into the Hadoop Distributed File System (HDFS). It can be used for dumping twitter data in Hadoop HDFS. It has a simple and flexible architecture based on streaming data flows; and is robust and fault tolerant with tuneable reliability mechanisms for failover and recovery. The Hive is a data warehouse tool that allows to store data into HDFS allows to process that using SQL. Hive is an excellent tool for analytical querying of historical data. While Pig uses procedural programming, language called Pig Latin to perform data processing and analytics. Pig Latin allows developers to insert their own code almost anywhere in the data pipeline which is useful for pipeline development which is accomplished using User Defined Functions UDFS that specify how data is loaded, stored and processed.

The procedure for twitter sentiments analysis are explained as follows

Step 1 : Creating Twitter Application

Open the website dev.twitter.com/apps in the Browser. Now we will see the website suggesting us to sign in. as shown in Fig. 5.11.1 (a). So, we sign into our twitter account then will be asked for creating a new app, so Click on Create New App as shown in Fig. 5.11.1 (b).

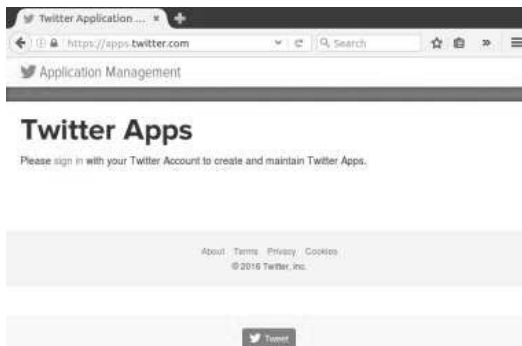


Fig. 5.11.1 (a) Signin to twitter

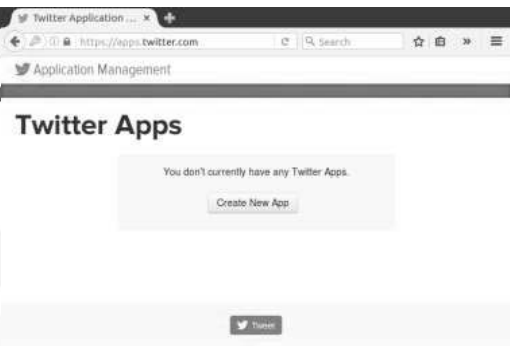


Fig. 5.11.1 (b) Create new app

Fill in all the required fields to make the application and use the website as google.com as shown in Fig. 5.11.2 then complete the form and click on Create your Twitter application .

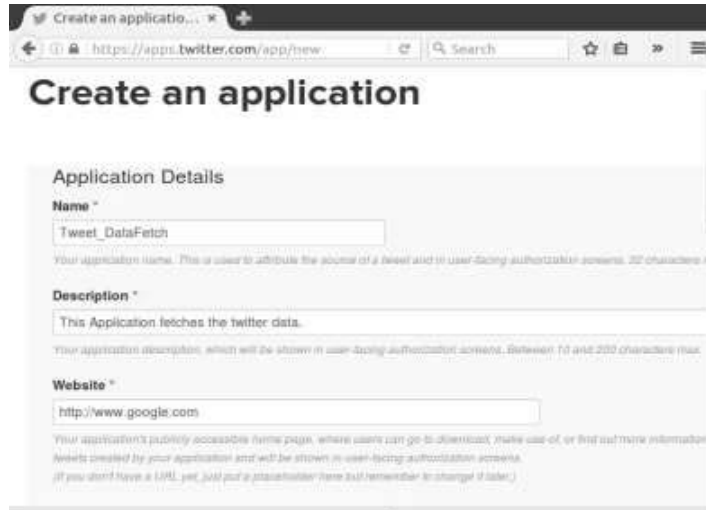
A screenshot of a web browser showing the 'Create an application' page on the Twitter developer portal. The page has a title 'Create an application' and a subtitle 'Application Details'. There are three main input fields: 'Name' with the value 'Tweet_DataFetch', 'Description' with the value 'This Application fetches the twitter data.', and 'Website' with the value 'http://www.google.com'. Below the website field, there is a small text block explaining that the website is the publicly accessible home page for the application. The browser's address bar shows 'https://apps.twitter.com/app/new'.

Fig. 5.11.2 : Create twitter application wizard

Step 2 : Open manage keys and access tokens from application settings and select my access token to create new token for the accessing the twitter data as shown in Fig. 5.11.3.

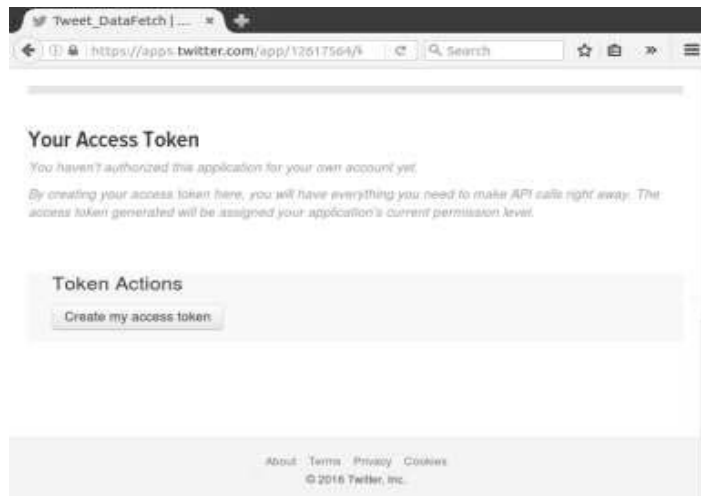
A screenshot of a web browser showing the 'Your Access Token' page on the Twitter developer portal. The page has a title 'Your Access Token' and a subtitle 'You haven't authorized this application for your own account yet.' Below the subtitle, there is a paragraph explaining that creating an access token will give the user everything needed to make API calls. There is a 'Token Actions' section with a button labeled 'Create my access token'. The browser's address bar shows 'https://apps.twitter.com/app/12617564/'.

Fig. 5.11.3 Twitter access token

Step 3 : After creating the access token, open **flume.conf** file in the directory **/usr/lib/flume/conf** and then change the following keys in the file. These keys will be

obtained from the page as shown in Fig. 5.11.4.

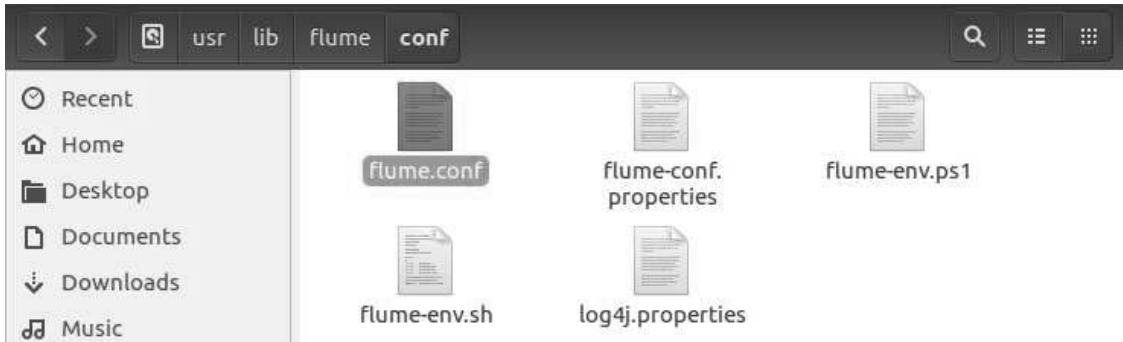


Fig. 5.11.4 : Flume Configuration File

Step 4 : Now changes needs to be done in flume.conf file for Access Token, Access Token Secret, Consumer Key (API Key) and Consumer Secret (API Secret) along with adding the keywords for search that we want to extract from twitter. In this example, we are extracting data on demonetization in India as shown in Fig. 5.11.5.

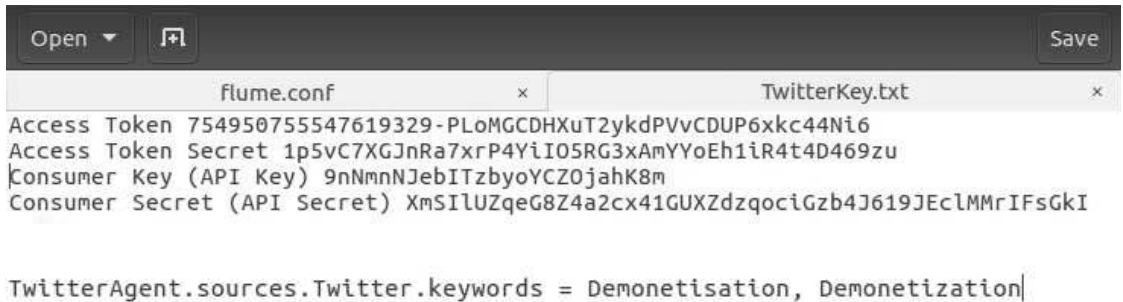


Fig. 5.11.5 : Configuration settings

Step 5 : Getting Data using Flume

After creating an application in Twitter developer's website, we can now access the Twitter data that we want to extract. The flume will be used to retrieve data from twitter. We will get everything in JSON format. The data will be fetched at given location comes from the Twitter and will be saved in the HDFS. After running the Flume, the Twitter data will automatically be saved into HDFS.

Following are the steps followed to collect and store dataset from Twitter into HDFS

1. Start terminal in ubuntu and execute following command to start flume agent
`$/usr/lib/flume/bin/flume-ng agent --conf ./conf/ -f /usr/lib/flume/conf/flume.conf -`

Dflume.root.logger=DEBUG,console -n TwitterAgent- Dtwitter4j.streamBaseURL=
https://stream.twitter.com/1.1/

2. By browsing HDFS, the list of extracted twitter data can be seen over the specified keyword in the conf file
3. The dataset provided by twitter can be seen as shown in Fig. 5.11.6.

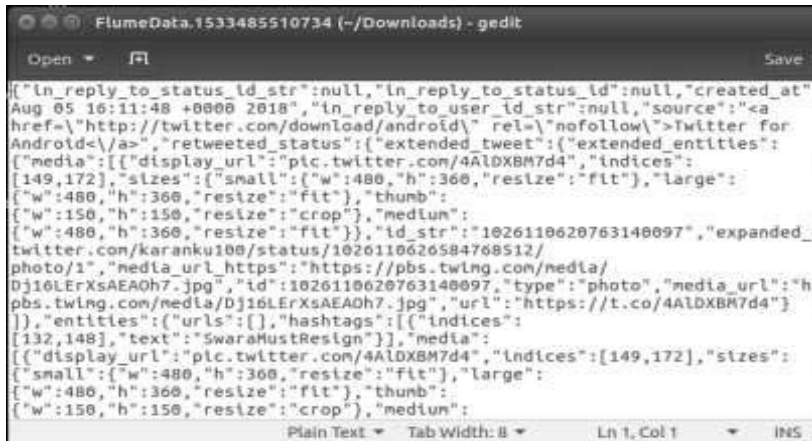


Fig. 5.11.6 Twitter dataset

Step 6 : Determining Popular Hash Tags using Hive

The tweets coming in from Twitter are in Json format, therefore we need to load the tweets into the Hive using json input format. So add jar file in to hive using following command.

```
ADD jar /usr/local/hive/lib/hive-serdes-1.0-SNAPSHOT.jar
```

After successfully adding the Jar file, create a Hive table to store the Twitter data. For calculating the hashtags, we need the tweet_id and hashtag_text, to be stored in Hive table. So, use following command to create a hive table.

```
CREATE EXTERNAL TABLE tweets (id BIGINT, entities  
STRUCT<hashtags:ARRAY<STRUCT<text:STRING>>>) ROW FORMAT SERDE  
'com.cloudera.hive.serde.JSONSerDe' LOCATION '/flumedir/data/tweets_raw';
```

Now, let's create another table which can store id and the hashtag text using the below command:

```
create table hashtag_word as select id as id, hashtag from hashtags LATERAL VIEW  
explode(words) w as hashtag;
```

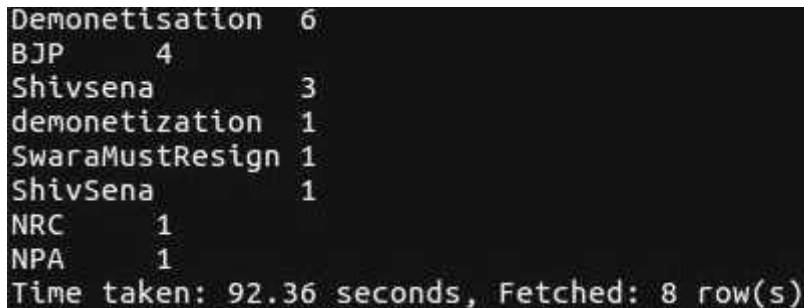
Now, let's use the query to calculate the number of times each hashtag has been repeated.

```
select hashtag, count(hashtag) as total_count from hashtag_word group by hashtag order by total_count desc;
```

Then execute following command to generate output

```
hive -f /hashtag.sql
```

The hashtag and the number of times it is repeated in the Twitter data will appear as a output as shown in Fig. 5.11.7.



The screenshot shows the output of a Hive query. It lists hashtags and their corresponding counts, sorted in descending order. The output is as follows:

Hashtag	Count
Demonetisation	6
BJP	4
Shivsena	3
demonetization	1
SwaraMustResign	1
ShivSena	1
NRC	1
NPA	1

At the bottom of the screenshot, it says: "Time taken: 92.36 seconds, Fetched: 8 row(s)"

Fig. 5.11.7 Hashtag output using hive

Step 7 : Load the tweets in to pig

The tweets are in nested Json format and consist of map data types. We need to load the tweets using JsonLoader which supports maps using following command

```
load_tweets = LOAD '/flumedir/data/tweets_raw/' USING  
com.twitter.elephantbird.pig.load.JsonLoader('-nestedLoad') AS myMap;
```

extract the id and the hashtag from the above tweets using following command

```
extract_details = FOREACH load_tweets GENERATE FLATTEN(myMap#'entities') as  
(m:map[]),FLATTEN(myMap#'id') as id;
```

Use following command to get formatted output

```
hash = foreach extract_details generate FLATTEN(m#'hashtags') as(tags:map[]), id as id;  
txt = foreach hash generate FLATTEN(tags#'text') as text, id;  
cnt = foreach grp generate group as hashtag_text, COUNT(txt.text) as hashtag_cnt;
```

The output of sentiment analysis using hashing is shown in Fig. 5.11.8.

```
(Demonetisation,6)
(BJP,4)
(Shivsena,3)
(SwaraMustResign,1)
(demonetization,1)
(ShivSena,1)
(NRC,1)
(NPA,1)
2018-08-12 20:06:23,464 [main] INFO  org.apache.pig.Main - Pig script completed in 2 minutes, 42 seconds and 608 milliseconds (162608 ms)
```

Fig. 5.11.8 : Popular Hashtags using Pig

Step 8 : Determining Average Rating of Tweets using hive

The command for creating a Hive table to store id and text of the tweets is as follows :

```
CREATE EXTERNAL TABLE load_tweets(id BIGINT, text STRING) ROW FORMAT SERDE
'com.cloudera.hive.serde.JSONSerDe' LOCATION '/flumedir/data/tweets_raw/';
```

Create a Split words

```
create table split_words as select id as id, split(text, ' ') as words from load_tweets;
```

Run the average rating query

```
select id, AVG(rating) as rating from word_join GROUP BY word_join.id order by rating
DESC;
```

Using above command, we have calculated the average rating of each tweet by using each word of the tweet and arranging the tweets in the descending order as per their rating. Now let us run sentiments.sql script as shown in Fig. 5.11.9.

```
ubuntu@ubuntu:~$ hive -f Desktop/sentiment.sql
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/usr/local/hive/lib/log4j-slf4j-impl-2.4.1.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/local/hadoop/share/hadoop/common/lib/slf4j-log4j12-1.7.10.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
```

Fig. 5.11.9 : Running sentiment.sql script

The output of average ratings by tweet ids is shown in Fig. 5.11.10 which shown the tweet_id and its rating.

1026138324669526016	2.0
1026138661006569472	2.0
1026137998365089792	2.0
1025045480533712897	2.0
1026138500242931712	2.0
1026139031543767041	2.0
1026138260718776320	0.5
1026138845262356481	0.5
1026139077924380674	0.5
1026137701446086658	0.5
1026138556983590912	0.5
1026138438926516225	0.5
1026138713733128193	0.5
1025045692450922497	0.0
1025045590957326336	0.0
1026138795131817984	-1.0
1026138734096408577	-1.0
1026138262782402560	-1.0
1026136868734230529	-1.0
1025045405590081536	-2.0
1026137886247333890	-2.0
1025045354352451584	-3.0
1026137672316645376	-3.0
1026138216620064768	NULL

Fig. 5.11.10 Average ratings by tweet ids

5.12 Big Data for E-Commerce and Blogs

The massive usage of Internet has transformed E-Commerce. The E-commerce customers now can access the wide range of products offered through E-Commerce websites. Therefore, in order to remain competitive and defend market, the E-Commerce firms formulates online marketing strategies based on real time data. As we know that, the E-commerce websites generates real time data in different formats and with massive in size which cannot be processed using traditional database management systems. So, E-Commerce firms are finding ways to extract meaningful information from larger datasets where data gets generated at greater velocity, different variety and at high volumes that are often referred to Big Data.

Therefore, Big data analytics came in to picture where it can be used to perform data acquisition, storage, processing and analytics of large datasets. As an effect of which, E-Commerce firms are investing huge on Big Data Analytics to empower them to take accurate and timely decisions.

5.12.1 Applications of Big Data Analytics for E-Commerce

Following are the Applications of big data analytics for E-Commerce

- a) **Predictive analytics** : The predictive analysis can allow E-commerce firms to predict the customer behavior. The Predictive analytics refers to the identification of events before they take place through the use of big data analytics. Therefore, Big Data analytics can enable you to predict the product demand, consumers behavior patterns & supply chain mechanics.
- b) **Dynamic pricing** : The Big data analytics in E-commerce can makes customers satisfactory by providing dynamic pricing. Dynamic pricing is required as majority of products compete on price offered with other sites. In order to attract new customers, E-Commerce companies must be vigilant and vibrant while setting competitive price for the products. E-Commerce firms need to actively influence the customer to buy at their site, which involves setting a competitive price.
- c) **Enhanced customer service** : The E-commerce firms can use Big Data to provide an exceptional customer service experience. Providing excellent customer service can lead firms to achieve competitive advantage.
- d) **Personalization** : Big data analytics offer personalized services, products, content and promotions for specific segments that makes better customer satisfaction with improved ROI.
- e) **Fraud detection** : By use of Big data in E-commerce, it can help in detecting fraudulent activities over the web. If fraud is detected in real-time, it leads to a speedier resolution and lower damage or loss. When fraud detection pattern is combined with Big Data powered real-time detection, the system gathers the required intelligence to detect and negate fraudulent practices. In addition, E-Commerce firms are able to identify fraud in real time by combining transaction data with customers" purchase history, web logs, social feed, and geospatial location data from Smartphone apps.
- f) **Supply chain visibility** : The Big data in E-commerce can allows you to track your goods ordered online while the goods are still in shipment has become the standard. Customers expect specific supply chain information, such as the exact availability, status and location of their orders. This involves intensive use of data infrastructure when E-Commerce firms have multiple third parties such as warehousing and transportation providers in their supply chain. Therefore, the use of Big data allows firms to build better models, which produce results with higher precision.

5.12.2 Big Data for Blogs

Unlike E-commerce, the big data analytics can also be used in Blogs to improve the blog sites. Nowadays, the blogs are found for every aspect like about travel, about pets, about food etc. The Big data analytics in blog can give

- Brief description of each blog from users perspective
- It can provide Alexa rank through which one can view users perspective about the blog and page views over the last three months to give a global traffic ranking and their update timing.
- The frequency with which the blog is updated.
- An interesting article from the site being reviewed.

5.13 Review of Basic Data Analytic Methods using R

R is a programming language used mostly for different types of data analysis and graphics which is available free under the General Public License (GNU). The following section explains the way to perform data analytics using R and to understand the flow of a basic R script to address an analytical problem. The following are the steps to execute

Step 1 : Import dataset

Suppose, we have a dataset in comma-separated-value file i.e. csv file which has data about annual sales in U.S. dollars for their customers. The `read.csv()` function is used to import the CSV file. The command for reading csv file is given below.

```
sales <- read.csv("c:/data/sales.csv")
```

Once the file has been imported, it is useful to examine the contents to ensure that the data was loaded properly as well as to become familiar with the data. In the example, the `head()` function, by default, displays the first six records of sales.

head(sales)

cust_id	sales_total	num_of_orders	gender
1	100001	500	F
2	100002	200	F
3	100003	708	M
4	100004	490	M
5	100005	721	F
6	100006	643	F

The `summary()` function provides some descriptive statistics, such as the mean and median, for each data column.

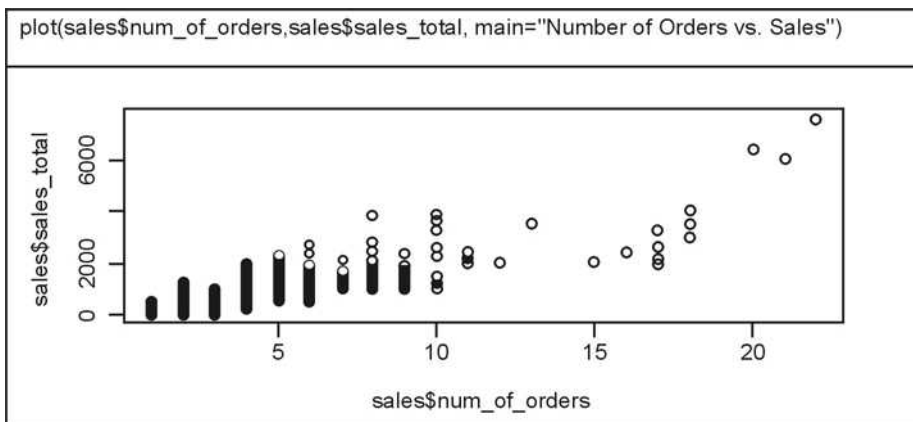
summary(sales)

```

cust_id sales_total num_of_orders gender
Min. :10001 Min. : 30.02 Min. : 1.000 F:5035
1st Qu.:10251 1st Qu.: 80.29 1st Qu.: 2.000 M:4965
Median :10501 Median : 151.65 Median : 2.000
Mean :10501 Mean : 249.46 Mean : 2.428
3rd Qu.:10750 3rd Qu.: 295.50 3rd Qu.: 3.000
Max. :11000 Max. :7606.09 Max. :22.000

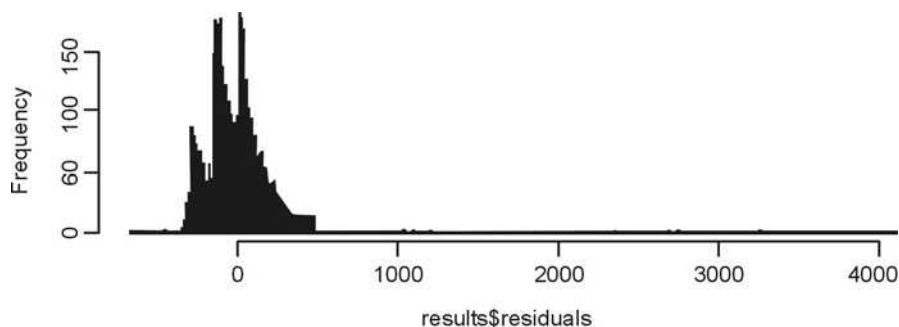
```

Plotting a dataset's contents can provide information about the relationships between the various columns.



The `hist()` function is used to plot the histogram for the dataset

hist(results\$residuals, breaks = 800)



Vectors are a basic building block for data in R. As seen previously, simple R variables are actually vectors. A vector can only consist of values in the same class. The tests for vectors can be conducted using the `is.vector()` function while The `array()` function can be used to restructure a vector as an array. The `matrix()` function is used to create matrix as shown below.

```
sales <- matrix(0, nrow = 3, ncol = 4)
```

```
sales
```

```
[,1] [,2] [,3] [,4]
```

```
[1,] 0 0 0 0
```

```
[2,] 0 0 0 0
```

```
[3,] 0 0 0 0
```